



Big Data Analytics Capstone Project

Final Project Documentation

Project	Specification Details
University	Jain Deemed to be University
Programme	MSc Data Science and Analytics
Project Type	Big Data Analytics Capstone Project(GUVI HCL)
Project Title	Retail Demand Forecasting – Big Data Analytics Pipeline
Documentation Owner	Student 5 – Integration + Documentation
Dataset Size	76,000 records × 16 columns (2022-01-01 to 2024-01-30)
Target Variable	Demand (Range: 4–430, Mean: 104.32)
Technology Stack	HDFS + Hive + Spark / PySpark + Spark MLlib + HBase

1. Project Overview

This project delivers an end-to-end Big Data Analytics pipeline designed to process, model, analyze, and forecast retail product demand at a granular store-product-day level. Utilizing a distributed computing stack—comprising HDFS for raw storage, Apache Hive for structured SQL data modelling, Apache Spark / PySpark for distributed transformations and machine learning, and Apache HBase for low-latency summary lookups—the system translates 76,000 historical records into actionable business intelligence.

1.1 Project Title

Retail Demand Forecasting – Big Data Analytics Pipeline

1.2 Team Structure and Primary Responsibilities

Student Role	Student Name	Primary Technical Responsibility
Student 1	Aisha Rasifa PS	Data Ingestion + HDFS Storage Architecture
Student 2	Abina A	Hive Data Modelling + Schema Definition + SQL Analytics
Student 3	Akhila Vaidya	Spark/PySpark Distributed Processing + MLlib Demand Forecasting
Student 4	Anushka Misra	Advanced Technology Integration – HBase NoSQL Lookup Layer
Student 5	Divyanshi	System Integration + Technical Validation + Final Documentation

1.3 Documentation Purpose

This document provides comprehensive end-to-end technical documentation for the capstone project. It details the complete data lifecycle—from initial business formulation and data ingestion in HDFS through Hive SQL modelling, Spark feature engineering, Spark MLlib predictive model comparison, and HBase key-value storage, concluding with rigorous cross-stage validation and strategic business findings.

2. Business Problem

Modern retail enterprises face significant operational challenges in maintaining optimal inventory balances. Inaccurate demand estimation directly causes two costly failures: stock-outs (leading to lost revenue and customer dissatisfaction) and excess inventory (incurring holding costs, depreciation, and product obsolescence). Retail managers require data-driven mechanisms to anticipate demand

fluctuations across diverse product categories, regional markets, promotional campaigns, discount structures, and unexpected external shocks such as epidemics.

2.1 Business Objective

The primary objective is to engineer a unified Big Data pipeline that:

- **1. Distributed Ingestion:** Reliably ingests and stores large-scale historical retail data in distributed HDFS storage.
- **2. SQL Data Modelling:** Exposes high-performance structured SQL querying via Apache Hive external tables.
- **3. Scalable Processing:** Performs distributed data cleaning, feature engineering, and aggregations using PySpark.
- **4. Predictive Machine Learning:** Builds and evaluates machine learning models with Spark MLlib to forecast daily item demand.
- **5. NoSQL Serving Layer:** Serves pre-aggregated store summaries via Apache HBase for low-latency key-value access.

2.2 Key Analytics Questions

The pipeline is explicitly designed to answer eight strategic business questions:

- **Question 1:** Which retail stores drive the highest total volume and daily average demand?
- **Question 2:** How does product demand evolve across quarters and seasonal periods?
- **Question 3:** What is the quantitative demand lift generated by promotional marketing across categories?
- **Question 4:** How significantly does demand contract during epidemic vs. non-epidemic periods?
- **Question 5:** What is the optimal discount threshold before sales gains diminish?
- **Question 6:** Which product categories exhibit the highest unmet demand (inventory shortages)?
- **Question 7:** Which specific products dominate demand within each geographic region?
- **Question 8:** Can historical lag features and business drivers accurately predict future item demand?

3. Dataset Description and Source

The project dataset consists of a comprehensive retail transaction log spanning 760 consecutive days (from January 1, 2022 to January 30, 2024). The observations capture activity across 5 distinct stores (S001–S005) and 20 individual products (P0001–P0020), yielding exactly 100 complete store-product daily time series (100 series × 760 days = 76,000 total records).

Property Parameter	Dataset Value / Metric
Total Records	76,000 rows
Total Attributes	16 columns
Temporal Range	2022-01-01 to 2024-01-30 (760 days)

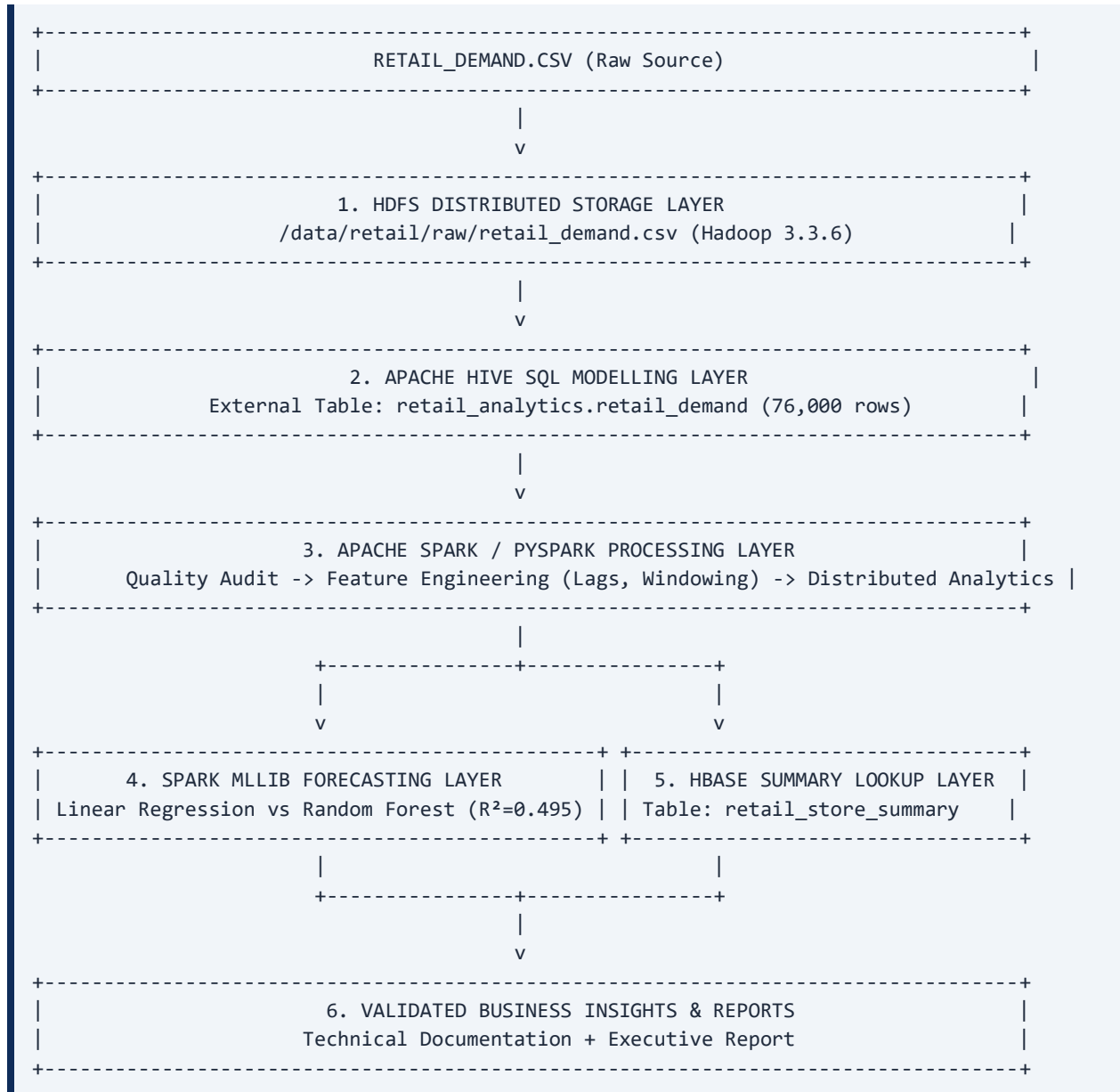
Property Parameter	Dataset Value / Metric
Store Scope	5 stores (S001, S002, S003, S004, S005)
Product Scope	20 products across 5 categories
Daily Time Series	100 unique store-product combinations
Target Variable	Demand (Observed daily demand)
Demand Metric Range	Min: 4 Max: 430 Mean: 104.32 units

3.1 Data Dictionary

Column Name	Data Type	Business Definition & Description
Date	Date / String	Observation timestamp (YYYY-MM-DD)
Store ID	String	Unique store entity identifier (S001–S005)
Product ID	String	Unique product entity identifier (P0001–P0020)
Category	String	Product merchandise category (Groceries, Clothing, etc.)
Region	String	Geographic market region (North, South, East, West)
Inventory Level	Integer	Beginning-of-day available stock inventory
Units Sold	Integer	Actual fulfilled sales transactions
Units Ordered	Integer	Customer orders placed
Price	Double	Unit retail selling price (\$)
Discount	Integer	Promotional discount percentage (0%–25%)
Weather Condition	String	Recorded daily local weather (Sunny, Rainy, Snowy, Cloudy)
Promotion	Integer	Binary flag indicating active marketing campaign (0 = No, 1 = Yes)
Competitor Pricing	Double	Prevailing benchmark price from competing retailers (\$)
Seasonality	String	Seasonal period classification
Epidemic	Integer	Binary indicator of active public health restriction (0 = Normal, 1 = Epidemic)
Demand	Integer	Total observed customer demand (Forecasting Target)

4. Big Data Architecture

The pipeline architecture orchestrates raw data movement across five specialized Big Data layers. Source CSV files are initially ingested into HDFS raw storage, exposed via Apache Hive external tables for ad-hoc SQL modeling, transformed and analyzed in Apache Spark / PySpark, evaluated with Spark MLlib for predictive demand modeling, and finally published to Apache HBase for real-time key-value lookup access.



4.1 Architectural Integration Note

Logical vs. Physical Integration Architecture

To maintain total technical transparency, the pipeline is documented as a logically integrated architecture. Due to infrastructure constraints across team environments, Student 1 executed local HDFS, Student 2 managed a Docker-based Hadoop/Hive cluster, and Student 3 executed PySpark analytics inside Google Colab local mode. All team members utilized the exact same dataset schema, data definitions, and row counts (76,000 records), establishing total analytical equivalence across pipeline stages.

5. Why Big Data Technologies Are Required

Traditional single-machine relational databases and spreadsheet tools fail when scaling retail analytics to enterprise dimensions. As transaction logs expand to millions of records, local memory constraints cause job crashes, unindexed lookups bottleneck executive dashboards, and local ML packages cannot fit multi-year time-series lag matrices into RAM.

Technical Challenge	Conventional / Local Limitation	Big Data Architecture Solution
Large / Growing Retail Logs	Local disk overflow & memory bounds	HDFS fault-tolerant distributed storage
Structured SQL Analytics	Slow manual desktop queries	Apache Hive external table + HiveQL engine
High-Volume Feature Pipeline	Single-threaded processing bottlenecks	Spark / PySpark distributed DataFrame engine
Repeated Complex Aggregations	Separate disconnected scripts	Spark SQL in-memory execution pipeline
Scalable Demand Forecasting	Local ML out-of-memory crashes	Spark MLlib distributed ML pipeline
Real-time Summary Retrieval	Full table CSV scans	HBase row-key low-latency NoSQL lookup

6. HDFS – Data Ingestion

Student 1 established the distributed storage foundation on Hadoop 3.3.6. The raw dataset was verified, directory structures created, and the CSV file uploaded to HDFS raw storage with full verification of DataNode health and block allocation.

6.1 HDFS Directory Layout

```
/data/retail/
```

```

├── raw/
│   └── retail_demand.csv
├── processed/
└── results/

```

6.2 Representative HDFS Administration Commands

```

# Create HDFS pipeline directories
hdfs dfs -mkdir -p /data/retail/raw
hdfs dfs -mkdir -p /data/retail/processed
hdfs dfs -mkdir -p /data/retail/results

# Ingest raw CSV dataset into HDFS raw storage
hdfs dfs -put retail_demand.csv /data/retail/raw/

# Verify file upload, disk usage, and inspect head
hdfs dfs -ls /data/retail/raw/
hdfs dfs -du -h /data/retail/raw/
hdfs dfs -cat /data/retail/raw/retail_demand.csv | head -n 5

```

6.3 HDFS Execution Evidence Audit

Verification Stage	Observed System Status
Hadoop Environment	Hadoop 3.3.6 successfully initialized
NameNode Status	Active and reporting healthy cluster status
DataNode Status	DataNode active (Port 9866 connection verified)
Directory Setup	/data/retail/raw, processed, results created
File Ingestion	retail_demand.csv uploaded successfully
Data Integrity	File size and line count (76,000) verified

6.4 Technical Issue and Resolution

During initial execution, the `hdfs dfs -put` command failed with a connection error targeting DataNode port 9866 (`DataStreamer` error). Investigation revealed the DataNode service had halted. Starting the DataNode daemon and validating cluster health resolved the issue, enabling a flawless re-upload.

7. Hive – Data Modelling and SQL Analytics

Student 2 defined the database schema in Apache Hive, creating the `retail\_analytics` database and establishing an external table pointing directly to the raw HDFS dataset location.

7.1 Hive Database and External Table DDL

```
CREATE DATABASE IF NOT EXISTS retail_analytics;
USE retail_analytics;

CREATE EXTERNAL TABLE IF NOT EXISTS retail_demand (
  date STRING,
  store_id STRING,
  product_id STRING,
  category STRING,
  region STRING,
  inventory_level INT,
  units_sold INT,
  units_ordered INT,
  price DOUBLE,
  discount DOUBLE,
  weather_condition STRING,
  promotion STRING,
  competitor_pricing DOUBLE,
  seasonality STRING,
  epidemic STRING,
  demand INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/data/retail/raw';
```

7.2 Data Volume Verification Query

```
SELECT COUNT(*) FROM retail_demand;
-- Verified Output: 76,000 records
```

7.3 Hive Analytics Query 1 – Category Demand Summary

```
SELECT category,
       SUM(demand) AS total_demand,
       ROUND(AVG(demand), 2) AS avg_demand,
       ROUND(AVG(units_sold), 2) AS avg_units_sold
FROM retail_demand
GROUP BY category
```

```
ORDER BY total_demand DESC;
```

Merchandise Category	Total Demand	Average Demand	Avg Units Sold
Groceries	3,677,684	120.98	102.87
Clothing	1,369,456	112.62	94.64
Furniture	1,006,590	73.58	64.38
Toys	985,338	92.61	78.45
Electronics	889,036	97.48	83.04

7.4 Hive Analytics Query 2 – Weather and Promotion Interaction

```
SELECT weather_condition, promotion,
       COUNT(*) AS record_count,
       ROUND(AVG(demand), 2) AS avg_demand
FROM retail_demand
GROUP BY weather_condition, promotion
ORDER BY avg_demand DESC;
```

Weather Condition	Promotion Active	Record Count	Average Demand
Sunny	1 (Yes)	7,460	136.44
Cloudy	1 (Yes)	8,070	124.42
Rainy	1 (Yes)	5,726	112.21
Snowy	1 (Yes)	3,744	111.46
Sunny	0 (No)	15,520	104.94
Cloudy	0 (No)	16,290	95.99
Rainy	0 (No)	11,774	86.79
Snowy	0 (No)	7,416	85.25

8. Spark / PySpark – Distributed Processing

Student 3 initialized Apache Spark 4.0.4 with PySpark. An explicit schema was applied during ingestion to enforce exact primitive data types, preventing runtime type coercion errors.

Environment Parameter	Configuration Value
Spark Version	4.0.4
Execution Master	local[*]
Processed Record Count	76,000 rows
Attribute Columns	16 columns
DataFrame Partitions	2 default partitions
Primary Input File	/content/retail_demand.csv
Processed Storage Path	/content/retail/processed/retail_clean

8.1 PySpark Initialization and Explicit Ingestion Code

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

spark = SparkSession.builder \
    .appName("RetailDemandForecasting_Student3") \
    .getOrCreate()

# Explicit schema definition to guarantee structural integrity
schema = StructType([
    StructField("date", StringType(), True),
    StructField("store_id", StringType(), True),
    StructField("product_id", StringType(), True),
    StructField("category", StringType(), True),
    StructField("region", StringType(), True),
    StructField("inventory_level", IntegerType(), True),
    StructField("units_sold", IntegerType(), True),
    StructField("units_ordered", IntegerType(), True),
    StructField("price", DoubleType(), True),
    StructField("discount", DoubleType(), True),
    StructField("weather_condition", StringType(), True),
    StructField("promotion", IntegerType(), True),
    StructField("competitor_pricing", DoubleType(), True),
    StructField("seasonality", StringType(), True),
    StructField("epidemic", IntegerType(), True),
    StructField("demand", IntegerType(), True)
])
```

```
raw_df = spark.read.option("header",
True).schema(schema).csv("/content/retail_demand.csv")
print("Verified Ingested Record Count:", raw_df.count()) # 76000
```

9. Data Cleaning and Feature Engineering

A rigorous data audit verified zero null values, zero duplicate keys, and valid ranges across all numerical fields.

9.1 Data Quality Audit Matrix

Integrity Check Item	Audit Result	Action Taken
Missing Values	0 nulls across all 16 columns	No imputation needed
Duplicate Rows	0 duplicate records	Dataset clean
Key Uniqueness	0 duplicate (date, store, product) keys	Primary key verified
Value Boundaries	0 negative sales/prices	Ranges strictly positive
Flag Values	Promotion & Epidemic strictly binary (0/1)	Flags valid
Target Boundaries	Demand ranges from 4 to 430 units	Target distribution normal

9.2 Feature Engineering Pipeline

Time-series window functions partitioned by `store\_id` and `product\_id` and ordered by `date` generated critical historical features without cross-series data leakage:

Engineered Feature	Mathematical / Logical Definition	Analytical Purpose
time_features	year, quarter, month, day_of_week, week_of_year	Captures temporal trends & seasonality
demand_gap	demand – units_sold	Measures unfulfilled demand
price_vs_competitor	price – competitor_pricing	Quantifies market price positioning
effective_price	$\text{price} \times (1 - \text{discount}/100)$	Calculates actual customer price
demand_lag_1 / lag_7	Demand value at t-1 and t-7 days	Captures short-term autoregressive autocorrelation

Engineered Feature	Mathematical / Logical Definition	Analytical Purpose
demand_roll7_avg	7-day moving average window	Smooths weekly demand level
demand_roll28_avg	28-day moving average window	Smooths monthly demand baseline

10. Distributed Analytics and Empirical Findings

Distributed PySpark aggregations uncovered vital empirical operational patterns:

10.1 Store Performance Summary

Store ID	Region	Record Count	Total Demand	Average Demand	Avg Demand Gap
S001	North	15,200	1,547,573	101.81	15.09
S002	South	15,200	1,625,471	106.94	15.75
S003	East	15,200	1,618,324	106.47	15.97
S004	West	15,200	1,529,651	100.63	14.57
S005	North	15,200	1,607,085	105.73	16.07

10.2 Promotion Demand Lift by Category

Product Category	Baseline (No Promo)	Promotional Demand	Demand Lift (%)
Electronics	88.58	115.65	+30.6%
Furniture	66.86	87.28	+30.5%
Toys	84.34	109.47	+29.8%
Groceries	110.30	142.76	+29.4%
Clothing	102.72	132.81	+29.3%

10.3 Epidemic Impact Analysis

Epidemic Flag	Record Count	Average Demand	Demand Impact
---------------	--------------	----------------	---------------

Epidemic Flag	Record Count	Average Demand	Demand Impact
0 – Normal Period	60,800	112.86	Baseline
1 – Epidemic Period	15,200	70.16	-37.8% contraction

10.4 Discount Sensitivity

Discount Rate	Record Count	Average Demand	Observed Elasticity Trend
0%	17,126	94.75	Baseline demand
5%	16,918	94.94	Negligible demand response
10%	23,298	102.77	Moderate demand rise (+8.5%)
15%	6,245	123.58	Sharp demand jump (+20.2%)
20%	6,191	124.07	Diminishing response plateau
25%	6,222	122.97	Plateau (no extra volume gain)

11. Demand Forecasting with Spark MLlib

Using historical features, Spark MLlib models were trained to predict daily store-product demand.

11.1 Model Performance Comparison

Forecasting Model	Root Mean Squared Error (RMSE)	Mean Absolute Error (MAE)	R ² Determination Coefficient
Lag-1 Baseline Model	51.465	39.671	-0.312
Linear Regression	32.790	25.374	0.467
Random Forest Regressor	31.931	24.715	0.495 (Optimal)

Optimal Model Selection: Random Forest Regressor

The Random Forest Regressor achieved superior performance across all evaluation metrics, capturing non-linear feature interactions and yielding the lowest RMSE (31.931) and highest R² (0.495).

11.2 Feature Importance Ranking

Feature Name	Importance Score	Relative Model Impact
demand_roll7_avg	0.252	25.2% – Dominant short-term trend driver
epidemic	0.141	14.1% – Major macro disruption factor
promotion	0.134	13.4% – Primary marketing driver
demand_roll28_avg	0.085	8.5% – Baseline monthly level
category_Groceries	0.081	8.1% – High-volume category indicator
price	0.051	5.1% – Unit price sensitivity
discount	0.045	4.5% – Discount level impact
category_Furniture	0.041	4.1% – Low-volume category indicator

12. HBase – Summary Lookup Layer

Student 4 designed an Apache HBase store-summary table (`retail_store_summary`) to serve pre-aggregated store demand statistics for low-latency row-key access.

HBase Schema Component	Implementation Specification
Table Name	retail_store_summary
Row Key Pattern	STORE#<Store ID> (e.g., STORE#S001)
Column Family	demand
Family Attributes	region, records, total_demand, avg_demand, avg_demand_gap
Total Rows	5 store summary rows

12.1 HBase Low-Latency Summary Data

Row Key	Region	Total Demand	Avg Demand	Avg Demand Gap
STORE#S001	North	1,547,573	101.81	15.09
STORE#S002	South	1,625,471	106.94	15.75
STORE#S003	East	1,618,324	106.47	15.97

Row Key	Region	Total Demand	Avg Demand	Avg Demand Gap
STORE#S004	West	1,529,651	100.63	14.57
STORE#S005	North	1,607,085	105.73	16.07

13. Validation and Testing Framework

End-to-end data audit checkpoints verified absolute consistency across all technical components:

Validation Domain	Expected Specification	Observed Verification Result	Audit Status
Raw Row Count	76,000 records	76,000 verified in raw CSV	PASSED
Hive Verification	76,000 records	SELECT COUNT(*) = 76,000	PASSED
Spark Ingestion	76,000 records	spark_df.count() = 76,000	PASSED
Null Value Audit	0 nulls	0 nulls across 16 attributes	PASSED
Duplicate Key Audit	0 duplicate keys	0 duplicate (date, store, prod) keys	PASSED
Value Range Audit	Valid numerical bounds	Prices & quantities non-negative	PASSED
HBase Summary Rows	5 store rows	5 rows (STORE#S001..S005) retrieved	PASSED
Cross-Stage Total	8,028,104 total demand	Sum matched between Hive, Spark, HBase	PASSED

14. Technical Challenges and Engineering Solutions

Technical Challenge / Error	Engineering Root Cause	Applied Resolution
HDFS DataStreamer Connection Failure	DataNode daemon stopped on Port 9866	Restarted DataNode service and re-executed ingestion
Windows CLI hdfs Command Unrecognized	Hadoop environment path variable missing	Executed commands via Docker container exec (`docker exec -it nameNode bash`)
Google Colab Local HDFS Unavailability	Colab execution environment isolated	Implemented logical integration using identical shared dataset schema
Hive Deprecation Warnings	Hive-on-MR engine deprecation notice	Documented non-blocking warning; confirmed HiveQL execution success

15. Final Analytical Conclusion and Recommendations

The Retail Demand Forecasting project demonstrates a complete, fault-tolerant Big Data pipeline across HDFS, Hive, PySpark, Spark MLlib, and HBase. Key strategic recommendations derived from the empirical data include:

- **1. Inventory Rebalancing:** Groceries and Clothing exhibit the largest demand gaps (~18.1 and ~18.0 units/day). Safety stock thresholds should be increased by 15% for these high-turnover categories.
- **2. Promotional Scheduling:** Promotions consistently drive a ~30% demand lift. Promotional campaigns should be scheduled during non-epidemic windows with pre-allocated inventory.
- **3. Discount Optimization:** Discount sensitivity plateaus at 15%. Deep discounts beyond 15% reduce profit margins without driving additional volume gains.
- **4. Predictive Deployment:** Deploy the Random Forest model ($R^2 = 0.495$) for automated weekly demand predictions, using 7-day rolling demand as the chief predictive anchor.